

Sicurezza Secondo Compitino

1 Introduzione

Safety engineering

prevenzione di danni "catastrofici" causati dal cattivo funzionamento di un sistema informatico.

Security engineering

prevenzione, difesa e recupero danni contro la possibilità di attacchi (più o meno diretti) che possono mettere a repentaglio i valori rappresentati da un sistema informatico.

Tutte le misure di safety sono anche misure di sicurezza, ma non è vero il viceversa.

Requisiti per la sicurezza:

- **Confidenzialità:**

- Capacità di controllare/assicurare **la privacy e la riservatezza** delle informazioni archiviate/trasmesse rispetto a soggetti non autorizzati;

- **Integrità:**

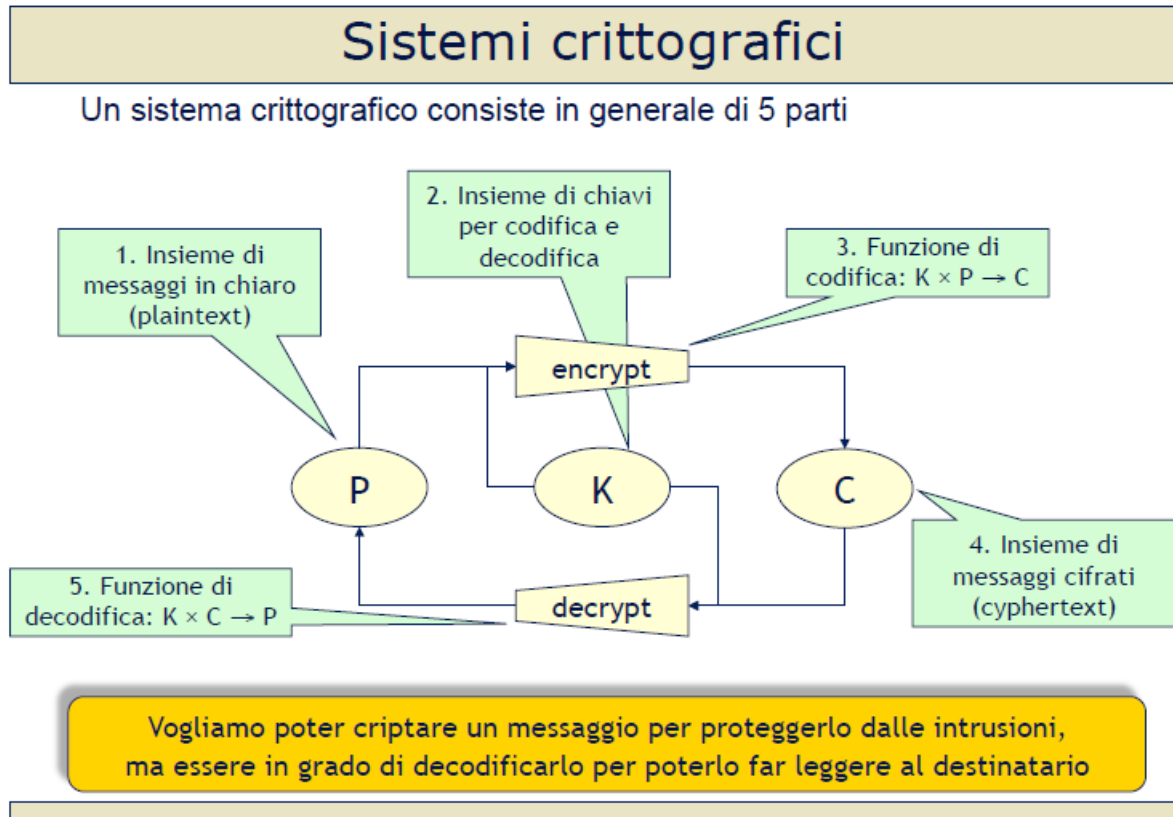
- Capacità di controllare/assicurare la:
 - Non modificabilità;
 - Attendibilità;da parte di soggetti non autorizzati delle informazioni memorizzate/trasmesse;

- **Disponibilità:**

- Capacità di controllare/assicurare la possibilità di usare risorse e dei servizi agli utenti autorizzati;

2 Crittografia

Disciplina che studia le tecniche per codificare/decodificare messaggi in modo da garantire la privacy della comunicazione fra due soggetti;



Crittografia:

- Schemi/Metodi/Algoritmi per la codifica sicura dei messaggi;

Crittoanalisi:

Obiettivi degli **Attacchi che tentano di "violare la crittografia"** attraverso lo studio di (insiemi di) messaggi criptati;

- Scoprire debolezze di un algoritmo, scoprire la chiave usata per cifrare un insieme di messaggi,

...

Un algoritmo crittografico è **violabile** se può essere **sempre** violato da un **crittoanalista**, a patto di poter disporre di tempo e risorse sufficienti;

Attacchi di "Forza Bruta":

Provare tutte le istanze di un insieme finito di schemi crittografici che potrebbero corrispondere al messaggio;

2.1 Algoritmi crittografici:

- Algoritmi simmetrici a chiave segreta;
- Algoritmi asimmetrici a chiave pubblica;
- Algoritmi di hashing (o message digesting);

Sistemi simmetrici (a chiave segreta)

Principio di funzionamento

Si usa la stessa chiave **k** sia per codifica che decodifica:

$$\text{decrypt}(k, \text{encrypt}(k, p)) = p$$

Per instaurare una comunicazione confidenziale, due soggetti devono conoscere una chiave **k** (**non nota a nessun altro**).

Supporto per i requisiti di sicurezza

Requisito	Si/No	Motivo
Confidenzialità	Sì	Solo chi conosce la chiave segreta può decodificare il messaggio
Integrità	Sì	una volta che il messaggio è stato crittato, non è possibile manometterlo prima della ricezione (il messaggio decodificato è esattamente lo stesso messaggio che era stato crittato)
Autenticazione e non ripudio	Sì	solo chi conosce la chiave segreta può essere mittente del messaggio

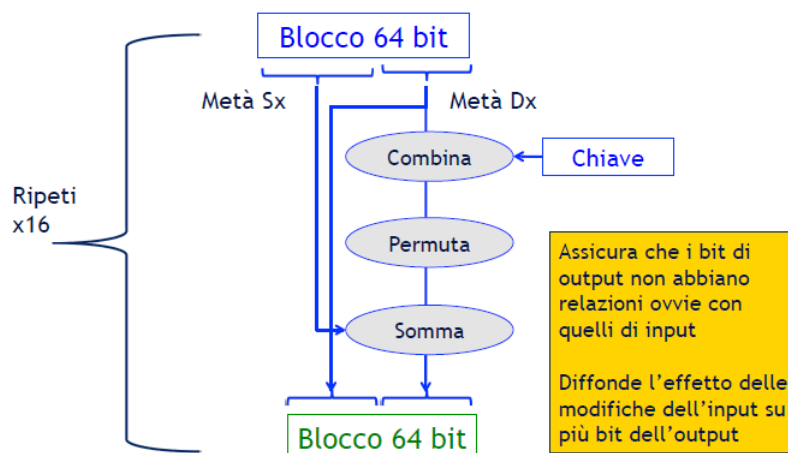
NB: Non è possibile garantire nessun requisito indipendentemente dagli altri

Quante chiavi segrete? **N** individui per comunicare in maniera sicura a coppie hanno bisogno di $\frac{n(n-1)}{2}$ chiavi (una per coppia)!

2.1.1 DES: Data Encryption Standard

Basato su confusione e diffusione dell'informazione:

- Codifica messaggi in blocchi da 64 bit;
- Applica 16 volte una **funzione combinatoria F** ad ogni blocco usando la chiave come uno dei parametri di F;



Sicurezza dei DES

- Non esiste la prova matematica della sicurezza di DES;
- Alternative a DES;
 - **Triple DES**: si cifra tre volte con 3 chiavi diverse. La sicurezza è pari all'uso di una chiave di 112 bit.

2.1.2 AES: Advanced Encryption Standard

- Basato su trasformazioni (sostituzione, scorrimento, mescolamento dei bit) applicabili efficientemente a blocchi di 128 bit;
- Utilizza chiavi da 128, 192 o 256 bit (estendibile);
- Numero di cicli fissato fra 10 e 14 (estendibile);

2.2 Sistemi asimmetrici (a chiave pubblica)

Ogni soggetto **A** possiede una **coppia di chiavi**:

- **k_pub** (chiave pubblica) è conosciuta da tutti;
- **k_priv** (chiave privata) non è condivisa con nessuno;

Un messaggio codificato con una delle chiavi, può essere decodificato **solo con la chiave corrispondente**:

$$\text{decrypt}(k_priv, \text{encrypt}(k_pub, p)) = p$$

$$\text{decrypt}(k_pub, \text{encrypt}(k_priv, p)) = p$$

Quante chiavi segrete? Per N individui c'è bisogno di N coppie di chiavi!

Supporto per i requisiti di sicurezza

Requisito	Sì/No	Motivo
Confidenzialità	Sì	Solo chi conosce la chiave segreta può decodificare il messaggio crittato con la corrispondente chiave pubblica
Integrità	Sì	una volta che il messaggio è stato crittato, con chiave pubblica o privata, non è possibile manometterlo prima della decodifica
Autenticazione e non ripudio	Sì	Solo chi conosce la chiave segreta può essere mittente di un messaggio crittato con la chiave segreta

NB: Ogni requisito può essere gestito indipendentemente dagli altri

Cifratura a chiave pubblica: RSA

- Si calcolano chiave pubblica e privata come funzioni di due numeri primi molto grandi (60..200 cifre ognuno);
- I numeri primi utilizzati sono scelti random e possono essere scartati dopo il calcolo delle chiavi (ma non devono essere rivelati);
- La sicurezza dell'algoritmo si basa sulla difficoltà di risolvere il problema di fattorizzare numeri molto grandi;
- Le chiavi possono essere usate intercambiabilmente per codificare e decodificare un messaggio;

2.3 Message Digest

- **Message digest = riassunto del messaggio**
 - Calcolo di una sorta di checksum crittografico
- **Bisogna usare un algoritmo a senso unico (non invertibile)**
 - Idealmente, ogni codice corrisponde a uno e un solo messaggio
 - Nella pratica questo non è possibile. Per esempio:
 - Codice 128 bit e messaggi 64 kb $\Rightarrow$ 1 codice ogni 65408 messaggi
 - Algoritmo non invertibile: dato il message digest di un messaggio, deve essere impossibile (o quantomeno difficilissimo) trovare un altro messaggio che abbia lo stesso message digest
- **Uso: integrità dei messaggi**
 - Supponiamo sia noto il codice riassuntivo MD del messaggio msg
 - Per verificare l'integrità del messaggio msg:
 - calcolo il message digest di msg
 - lo confrontarlo con quello noto MD
 - se sono uguali, allora è molto probabile che il messaggio non sia stato modificato

2.3.1 MD5

Applica una **trasformazione combinatoria complessa** a blocchi di 512 bit del messaggio e genera un output a 128 bit.

Supporto per i requisiti di sicurezza

Requisito	Sì/No	Motivo
Confidenzialità	No	Nessuno può decodificare il codice MD di un messaggio (non invertibilità)
Integrità	Sì	La coppia <messaggio, codice MD> permette di stabilire che il messaggio non sia stato manomesso
Autenticazione e non ripudio	No	Chiunque può aver generato il codice MD di un dato messaggio

DES e **MD5** sono parecchi ordini di grandezza più veloci di **RSA**. Se implementati in hardware (**chip dedicato**), DES e MD5 raggiungono velocità misurabili in **Gb/s**, mentre RSA in **Kb/s**; Tipicamente si usa **RSA** per cifrare **piccole quantità di dati**.

Requisiti di **Confidenzialità**: **RSA + DES**

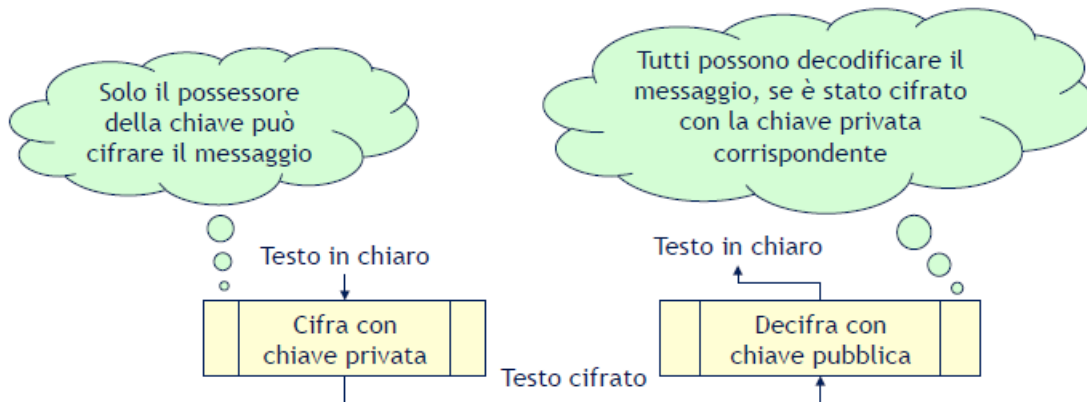
- Si cifra con RSA una chiave segreta casuale ("**chiave di sessione**");
- Una volta scambiata la chiave di sessione, la si usa come **chiave segreta di DES**;

Requisiti di **Integrità + Autenticità**: **RSA + MD5**

- Attraverso la realizzazione di firme digitali;

2.4 Firme Digitali

- La crittografia a chiave pubblica soddisfa i requisiti necessari per la firma digitale (autenticazione e non-ripudio)
- Deve essere usata "al contrario"
 - Si cifra il messaggio con la chiave privata (solo il possessore)
 - Lo si decifra con la chiave pubblica (tutti)



- Cifrare un messaggio di grosse dimensioni (p.es. con RSA) può essere poco efficiente, però...
 - si può firmare un piccolo pezzo del messaggio,
 - a condizione che non sia possibile contraffare un documento dopo che è stato firmato
- Soluzione: si firma il message digest del messaggio
 - Il destinatario riceve il messaggio M in chiaro e la firma F cifrata dal mittente con la sua chiave privata
 - Il destinatario autentica F (cioè la decifra con la chiave pubblica del mittente) e come risultato ottiene il message digest del messaggio originale
 - Il destinatario confronta il message digest del messaggio ricevuto con quello decifrato dalla firma in maniera da verificare l'integrità del messaggio

- In un sistema crittografico a chiave pubblica, uso la chiave pubblica per:

- ➔ autenticare provenienza e integrità di un messaggio
- ➔ mandare messaggi confidenziali

- Requisito

- ➔ La chiave pubblica deve corrispondere effettivamente all'entità che voglio autenticare o a cui voglio mandare messaggi confidenziali

- Problema

- ➔ Come faccio a stabilire con sicurezza chi è il proprietario di una chiave pubblica?

Certificati X.509

Un certificato garantisce la veridicità di una chiave pubblica rispetto ad una certa entità (persona o azienda). Un certificato è emesso da un'autorità di certificazione "fidata e autorizzata".

X.509 è uno standard per emettere certificati che garantiscano l'identità associata ad una chiave pubblica;

Un certificato X.509 contiene:

- Nome di un'entità;
- Chiave pubblica dell'entità certificata;
- Nome dell'autorità di certificazione che lo rilascia;
- Firma digitale da parte dell'autorità di certificazione;
- Se si dispone della chiave pubblica dell'autorità di certificazione si può verificare la veridicità e l'integrità di un certificato;
- Se si ha fiducia nell'autorità di certificazione, ci si fida che la chiave pubblica e l'entità riportate nel certificato corrispondano effettivamente;

3 Sicurezza nelle Applicazioni

Schemi di attacco:

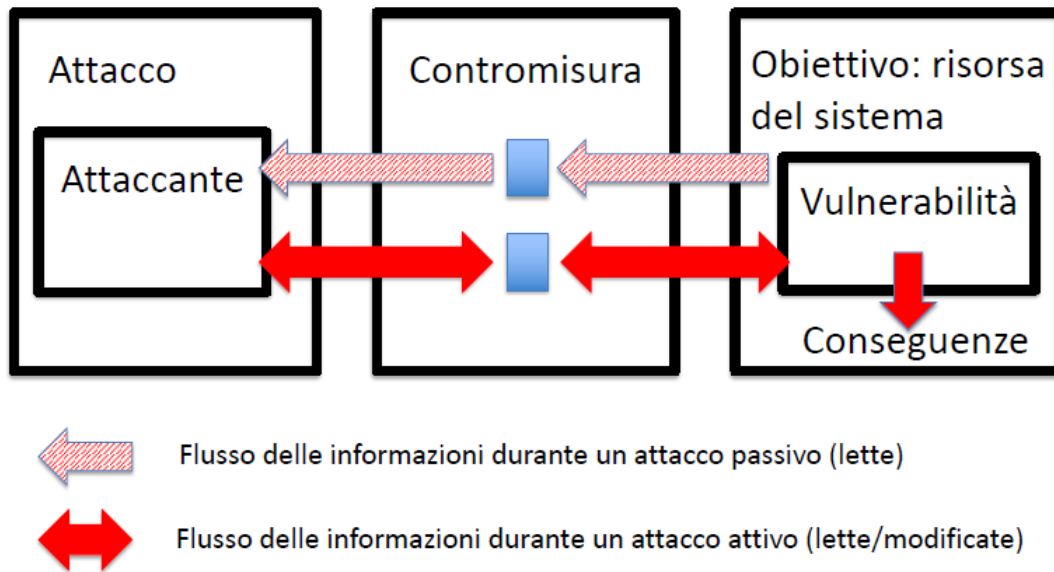
- Tassonomia (**NON SONO OGGETTO DI ESAME**);
- XSS;
- SQL injection;
- Buffer overflow;
- Integer overflow;
- Null pointer dereference;

Il Glossario della Sicurezza

- **Security policy**: Un insieme di regole che determinano come un sistema o un'organizzazione protegge risorse critiche e sensibili;
- **Weakness**: un tipo di errore nel progetto, implementazione o operazione di un sistema software che, in condizioni appropriate, può portare all'introduzione di vulnerabilità nel sistema;
- **Vulnerabilità**: l'occorrenza di una weakness (o più di una) in un sistema software, che possono essere sfruttate da una terza parte per violare la security policy del sistema;
- **Minaccia** (threat): causa potenziale di violazione della security policy di un sistema, che può causare un danno (accesso non autorizzato, rivelazione/modifica/distruzione di informazione, disabilitazione di un servizio);
- **Attacco** (attack): un'azione o una tipologia (schema di attacco, attack pattern) di azione intenzionale volta alla violazione della security policy di un sistema;
- **Attaccante** (attacker): un'entità che effettua un attacco;
- **Exploit**: una tecnica che permette di sfruttare una vulnerabilità per violare la security policy di un sistema;
- **Contromisura** (countermeasure): un'azione o tecnica che elimina, prevenisce, mitiga o individua una vulnerabilità, minaccia o attacco;

Caratteristiche attacco

- **Tipologia**
 - **Attivo**: altera le risorse del sistema e le sue operazioni;
 - **Passivo**: acquisisce informazioni dal sistema;
- **Point of initiation**
 - **Interno**: attaccante ha accesso alle risorse ma le vuole usare in modo differente (e.g. scrivere un file);
 - **Esterno**: attaccante risiede al di fuori del perimetro di sicurezza del sistema;
- **Method of delivery**
 - **Diretto**: attacca il sistema target direttamente;
 - **Indiretto**: utilizza un terzo sistema per accedere al target o per amplificare l'attacco;



Diverse organizzazioni si occupano di collezionare e riportare **weakness**, schemi di attacco, vulnerabilità di software noti. Le più note sono:

- **MITRE**: organizzazione finanziata dal dipartimento della difesa americano. Mantiene:
 - Common Weakness Enumeration (**CWE**)
 - Common Attack Pattern Enumeration and Classification (**CAPEC**)
 - Common Vulnerability and Exposure (**CVE**)
- **Web Application Security Consortium (WASC)**: non legata ad organizzazioni governative
 - **WASC Threat Classification** classifica schemi di attacco e weaknesses

Common Weakness Enumeration (CWE)

- Elenco formale di weakness;
- Destinatari: sviluppatori, esperti di sicurezza;
- Scopo:
 - Fornire un dizionario comune per descrivere le weakness nel software;
 - Utile per misurare le potenzialità degli strumenti software di analisi/verifica;

Common Attack Pattern Enumeration and Classification (CAPEC)

- Catalogo di schemi (pattern) di attacco;
- CWE descrive le weakness che possono essere presenti in un software;
- CAPEC descrive come queste sono tipicamente sfruttate;

Common Vulnerability and Exposures (CVE)

- Elenco aggiornato di vulnerabilità che riguardano software molto diffusi;

3.1 Vulnerabilità di tipo convalida degli input incompleta: XSS, SQL Injection, Heartbleed, Buffer overflow

3.1.1 XSS: Cross Site Scripting

Attacco di tipo **code injection**.

Si verifica quando un attaccante riesce a eseguire del codice malevolo all'interno del browser di un utente sfruttando una vulnerabilità di un sito web ignaro dell'attacco.

Il codice malevolo è implementato in un linguaggio di scripting, ad esempio Javascript;

Perché un attaccante sceglie di sfruttare un linguaggio di scripting?

- Ha accesso ad informazioni sensibili, e.g. cookies;
- Può inviare **richieste HTTP** contenenti qualsiasi dato verso una qualsiasi destinazione tramite chiamate a **XMLHttpRequest** o altri meccanismi;
- Può modificare HTML della pagina utilizzando funzionalità del **DOM**;

Conseguenze:

- **Furto di cookie**

- l'attaccante può accedere ai cookie tramite la variabile `document.cookie`
- Inviarli al server
- Usarli per estrarre dati sensibili, e.g. session IDs.

- **Keylogging**

- L'attaccante può registrare un keyboard event listener (tramite chiamata a `addEventListener`);
- Quindi registrare ed inviare ad un proprio server tutti i caratteri premuti (potenzialmente password);

- **Phishing**

- L'attaccante può creare un fake login;

Tipologie:

- **Persistent XSS**

- Il codice malevolo risiede nel DB del sito web;

- **Reflected XSS**

- Il codice malevolo risiede nella richiesta HTTP della vittima;

- **DOM-based XSS**

- La vulnerabilità risiede nel codice eseguito nel client, non nel server;

Persistent XSS

- L'attaccante utilizza una form del sito web per inserire del codice malevolo in una pagina web;
- La vittima richiede una pagina dal sito web;
- Il sito web (ignaro) include nella pagina web la stringa inserita dall'attaccante ed invia la risposta alla vittima;
- Il browser della vittima esegue lo script malevolo contenuto nella pagina web;

Reflected XSS

- L'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima;
- La vittima clicca sull'URL;
- Il sito web invia la risposta ed include la stringa malevola nella risposta;
- Il browser della vittima esegue il codice malevolo;

DOM-based XSS

- Sfrutta scripting dinamico (capacità di una pagina web di creare il contenuto a runtime);
- L'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima;
- La vittima clicca sull'URL;
- Il sito web riceve la richiesta, ed invia la risposta senza includere il codice malevolo;
- La pagina web creata dal sito è dinamica, e il codice Javascript incluso nella pagina:
 - Estrae il codice malevolo dall'URL;
 - Lo aggiunge dinamicamente alla pagina;
- Il browser dell'utente interpreta lo script malevolo;

Guarda meglio le slide per esempi e altro

3.1.2 SQL Injection

Attacco di tipo **code injection**.

Consiste nell'inserimento nel sistema di una query SQL attraverso l'input utente al fine di modificare l'esecuzione di comandi SQL predefiniti.

Un attacco eseguito con successo può:

- Leggere/modificare dati sensibili da un DB;
- Eseguire operazioni di amministrazione del DB (shutdown);
- Recuperare il contenuto di un file presente nel filesystem del DBMS;

I problemi di SQL injection si possono manifestare in tutti i programmi che fanno uso di un DBMS. Storicamente sfruttati nel caso di web applications scritte con:

- PHP;
- Python;
- Perl;
- ...

Librerie di encoding per ovviare al problema disponibili per i maggiori linguaggi.

3.1.3 Heartbleed

Vulnerabilità specifica che riguarda **OpenSSL 1.0.1**.

OpenSSL è una Cryptographic Software Library, usata da software molto diffusi, e.g. **web server Apache**.

Questa vulnerabilità dipende da un mancato controllo di un input.

3.1.4 Buffer overflow

Il termine buffer overflow indica una situazione in cui una scrittura su un buffer esce dai limiti del buffer. In questi casi i valori delle aree di memoria adiacenti vengono sovrascritti.

Descrizione del difetto che causa un buffer overflow:

- Il programma copia il buffer a nel buffer b;
- Il buffer a contiene dati che sono stati ricevuti da un utente;
- **Non ci sono controlli sul fatto che b sia sufficientemente grande da contenere i dati;**
- Questo difetto può essere sfruttato per effettuare un attacco;

Tramite **buffer overflow** possiamo sovrascrivere ogni area di memoria "raggiungibile" dal buffer, ad esempio il **return address**.

Shellcode

Lo shellcode è un piccolo programma che utilizza la chiamata di sistema **execve** per sostituire il processo corrente con uno nuovo, in questo caso la shell. All'interno del file (e quindi in memoria) lo shellcode è contornato da **NOP** (istruzioni assembler che "non fanno nulla").

In questo modo il "salto" può essere impreciso (può essere utile se lo stack cresce in maniera non deterministica);

Lo shellcode viene di solito usato quando la vulnerabilità riguarda un servizio che viene eseguito con privilegi di amministratore di sistema:

- In questo modo lo **shellcode** sarà eseguito con i privilegi di amministratore;
- E' utile quando l'attacco è sferrato dall'interno, ad esempio se l'attaccante possiede un account utente sul server;

Heap Buffer Overflow

Un buffer overflow può riguardare anche variabili sullo heap:

Non può essere sfruttato per **riscrivere il return address**, Ma si possono riscrivere altri elementi, ad esempio **puntatori a funzione**.

Protezioni: Canary Value

Istruzioni inserite dal compilatore a compile time.

Ad ogni invocazione di funzione viene inserito un valore di controllo (**canary value**) sullo **stack**, dopo il **BP**.

Il canary value è tipicamente un valore random, generato in fase di inizializzazione del programma e viene controllato all'uscita dalla funzione.

Il canary value **deve essere identico al valore inserito in precedenza** (il sistema lo ricorda in quanto contenuto in un area specifica di memoria, "lontana" dallo stack).

Se il controllo fallisce il programma viene terminato con un **messaggio di errore**, come segue:

- **** stack smashing detected ***: ./myProgram terminated*

Opzioni di protezione fornite dal compilatore GCC -fstack-protector

- Genera codice aggiuntivo per verificare la presenza di buffer overflow;
- Aggiunge un canary value (guard variable) **SOLO alle funzioni che possono presentare vulnerabilità con maggiore probabilità**, ad esempio:
 - Funzioni che invocano la funzione "alloca" (funzione di sistema che alloca un certo numero di bytes sullo stack frame);
 - Funzioni con buffers più grandi di 8 bytes;

-fstack-protector-all

- Protegge **tutte** le funzioni;
- Più costoso;

-fstack-protector-strong (Da GCC 4.9):

Compromesso tra completezza e overhead, attivato in quelle funzioni per cui:

- l'indirizzo di una variabile locale viene usato nella parte destra di un assegnamento, o come parametro passato a funzione;
- dichiara variabili locali di tipo **array**;
- usa **"register local variables"**;

Address Space Layout Randomization (ASLR)

Contromisura implementata nei moderni sistemi operativi.

Alcuni indirizzi di memoria del virtual address space **cambiano da un'esecuzione ad un'altra**. Rende più difficile la **scrittura degli exploit**, ad esempio diventa difficile "indovinare" l'indirizzo da utilizzare per riscrivere il "return address".

Executable Space Protection

Termine che indica la possibilità per il sistema operativo di distinguere aree di memoria contenente codice da eseguire da aree di memoria contenente dati.

Spesso richiede il supporto sia da parte del sistema operativo che parte dell'hardware.

Nei processori è supportata attraverso **NX bit**: l'ultimo bit di un indirizzo della page table viene usato per indicare se una pagina contiene **codice eseguibile** o meno.

JIT Spraying

Alcuni linguaggi interpretati fanno uso di compilatori **Just-in-time** (JIT) che interpretano uno script di input e generano al volo del codice eseguibile in linguaggio macchina.

Il codice generato dal **JIT compiler** deve essere per forza posizionato su **pagine eseguibili**.

Attacchi tramite **JIT** (detti **JIT spraying**):

- Alcuni attacchi sfruttano la presenza del JIT compiler per **generare codice malevolo** (ad esempio a partire da script Javascript);
- Quindi **sfruttano buffer overflow** (o use after free) per redirezionare l'esecuzione verso il nuovo codice;

Buffer Overflows in Java

Gli array dei programmi Java sono protetti da buffer overflow. Tuttavia tutto il codice nativo intorno ad un programma Java può essere affetto da vulnerabilità di questo tipo:

- La JVM è scritta in C++ ;
- In un programma Java si **possono invocare librerie implementate in linguaggio nativo** (tramite **JNI**);
- Il compilatore JIT di Java può presentare difetti che causano l'**assenza di controlli su array**;

3.2 Attacchi che sfruttano Code quality: Null Pointer Dereference

Il valore **NULL** viene tipicamente assegnato alle variabili puntatore dopo la loro dichiarazione per indicare che la **variabile è vuota** (non punta a niente).

Per il **sistema operativo** rappresenta un **valore non valido**, ogni tentativo di accedere a questa memoria porta ad un errore detto **segmentation fault** (causa un errore a runtime).

Per l'**hardware** è un indirizzo come gli altri, è l'**indirizzo 0**, che rappresenta la **prima cella** di memoria.

I processi possono scegliere i propri indirizzi di memoria virtuali tramite chiamata alla funzione **mmap**.

Se assegniamo con mmap la pagina 0 all'interno dello spazio di indirizzo del processo non avremo più Segmentation Faults perchè l'indirizzo 0 (NULL) diventa un **indirizzo valido**.

NOTA: La **Segmentation Fault** lanciata dal sistema quando il programma dereferenzia un puntatore NULL **serve per evitare errori più difficili da notare**, che potrebbero avere conseguenze peggiori.

- Es. inserimento valori sporchi. Una lettura dall'indirizzo 0 infatti spesso corrisponde al tentativo di lettura di una struttura dati non allocata/inizializzata.

Come viene sfruttato?

- Per iniettare dati malevoli **all'interno di un programma in esecuzione**;
- Di solito come "trampolino" per **eseguire del codice malevolo in modalità kernel**;

Spazio indirizzi del Kernel

Il kernel ha un proprio spazio degli indirizzi, tuttavia i processi hanno bisogno di eseguire delle **kernel system call** per effettuare operazioni "**protette**", come ad esempio leggere un file.

Il context switch tra processi e kernel è **costoso**:

- per gestire al meglio il context switch tra un processo ed il kernel il codice del kernel è **mappato nello spazio di memoria del processo**;
- i processi non possono eseguire il codice del kernel "saltando" nel suo spazio di memoria, perchè esiste un meccanismo hardware detto memory protection;
- tuttavia ci sono degli **entry point** che fanno sì che un processo possa eseguire operazioni privilegiate tipicamente riservate al kernel, queste sono le **system call** (necessarie ad esempio per scrivere un file);

Difetti Sistema Operativo

Una Null Pointer Dereference nel codice del sistema operativo tipicamente causa un **crash dell'intero sistema**, ma spesso può venire sfruttata da un attaccante per eseguire istruzioni con i permessi di **amministratore del sistema**.

Lo stesso problema si presenta anche quando **il codice di un modulo del sistema operativo** (ad esempio un device driver) **presenta una null pointer dereference**.

Particolarmente **grave è quando il puntatore è un puntatore a funzione**.

mmap può esser usata per eseguire funzioni puntate da puntatori NULL

Se mappiamo l'indirizzo 0 all'interno dello spazio di indirizzi del processo possiamo far sì che un **puntatore a funzione posto a NULL esegua una funzione da noi desiderata**.

Questa caratteristica viene sfruttata dagli attacchi basati su NULL pointer dereference.

- Esegue mmap
- Scrive all'indirizzo zero una funzione creata ad hoc (ad esempio uno shellcode)
- Esegue una chiamata di sistema su dei parametri opportunamente configurati tale che
 - Un puntatore a funzione è impostato a NULL
 - Il kernel usa il puntatore per invocare la funzione, eseguendo ciò che sta all'indirizzo zero, cioè il codice dell'attaccante
- Il codice malevolo viene eseguito in contesto kernel, e di solito:
 - assegna i permessi di root al processo corrente (il kernel può assegnare permessi di root ad un qualsiasi processo)
 - esegue dei comandi nel processo corrente (che ormai ha i permessi di root)
 - questi comandi corrispondono spesso ad uno shellcode
 - potrebbero essere altri comandi

2 ESEMPI SU SLIDE CHE NON AVEVO VOGLIA DI SCRIVERE

3.3 Java Secure Coding Guidelines

Vengono presentate diverse linee guida raggruppabili in due categorie: • **Domain Dependent Guidelines**

- Linee guida per cui è difficile definire dei controlli automatici;
- **Domain Independent Guidelines**
 - Linee guida che si riferiscono alla struttura del codice per cui è automatizzabile il controllo;
 - **FindBugs** ad esempio è un tool che controlla alcune linee guida;

Domain Dependent Guidelines

Limit the lifetime of sensitive data

Dati sensibili potrebbero essere ispezionati da un attaccante se l'applicazione;

- usa oggetti per memorizzare dati sensibili, e gli oggetti non sono ripuliti/garbage collected;
- ha pagine di memoria che vengono scaricate (swap) su disco;
- mostra i dati sensibili in messaggi di debugging
- **Soluzione:** ripulire i dati:
 - Es. sovrascrivere la stringa usata per ricevere la pwd dell'utente;

Store Passwords using Hash Functions

Password salvate in plain text potrebbero essere osservate da un attaccante. **Utilizzare "Hash functions"**.

Minimizzare la visibilità delle Variabili

NO

```
public static void main(String args[]){  
    int i;  
    for( i = 0; i < args.length; i++ ){  
        /* Do something*/  
    }  
}
```

SI

```
public static void main(String args[]){  
    for( int i = 0; i < args.length; i++ ){  
        /* Do something*/  
    }  
}
```

Creare classi non serializzabili (se necessario)

La serializzazione in alcuni contesti potrebbe essere pericolosa perchè **permette ad un attaccante di accedere allo stato interno degli oggetti.**

Rendere le classi non serializzabili anche non deserializzabili

Un attaccante potrebbe creare un oggetto ad hoc che può essere deserializzato, pure per una classe non serializzabile -

Creare classi non clonabili(se necessario)

Un attaccante potrebbe evitare di eseguire il costruttore della vostra classe (che potrebbe contenere security checks).

Ad esempio può creare una sottoclasse che si limita ad implementare il metodo "clone".

Findbugs Domain Independent Guidelines

Sono solo un paio di pagine nelle slide.

4 Sicurezza nei sistemi operativi

Autenticazione

Obiettivo: Evitare l'accesso di soggetti non autorizzati alle risorse del sistema.

Metodi di Autenticazione

- Dimostrazione di conoscenza
 - PIN (Personal Identification Number)
 - Password
 - Questionari
- Dimostrazione di possesso
 - Chiavi fisiche
 - Smartcard d'identificazione
- Caratteristiche personali uniche (biometria)
 - Impronte digitali
 - Immagine della retina
 - Riconoscimento vocale

Attacchi alle password

L'aggressore proverà:

- Password assente
- Password uguale/derivata a UserID o informazioni note per l'utente
- Lista di nomi comuni, breve vocabolario per una o più lingue
- Breve vocabolario con maiuscole e sostituzioni (i=1, o=0, a=@, ...)
- Dizionario completo per una o più lingue
- Attacco forza bruta con numeri e lettere minuscole o maiuscole
- Attacco forza bruta con set completo di caratteri

Inoltre sono possibili scelte mirate se si hanno informazioni sul meccanismo di generazione delle password.

Comportamenti scorretti degli utenti nella scelta delle password

- Password troppo semplici, ovvero facilmente identificabili con dizionari di password;
- Riduzione del set di caratteri (semplifica gli attacchi di **forza bruta**);
- Scrivere o condividere la password (mette a repentaglio la segretezza della password);

Password file criptati

Applicazione della crittografia unidirezionale (MD5/SHA-1, ...)

Uso di **"salt"** per ovviare al problema di utenti che possono scegliere password uguali

- Salt = numero di 12 bit random
- Il salt viene memorizzato nel file di password
- La password viene memorizzata come **hash(pw + salt)**

Multiple point of failure: Il sistema operativo protegge il file delle password oltre a mantenere le password criptate

One Time Password (OTP)

La password è una funzione matematica $f(x)$ che l'utente sa calcolare.

Per l'autenticazione:

- il sistema propone un valore a caso X
- il richiedente deve rispondere con il valore corretto $F(X)$

La funzione può essere:

- Una funzione "segreta" di numeri o stringhe;
- Un generatore di numeri disponibile all'utente e al sistema;
- Una funzione crittografica:
 - Es: $f(\text{encrypt}(x)) = \text{encrypt}(\text{decrypt}(\text{encrypt}(x)) + \dots)$

Processo di Autenticazione

Un sistema di autenticazione ben progettato:

- Non dovrebbe fornire dettagli su nomi utente e sistema prima che l'autenticazione sia avvenuta con successo
- Può essere intenzionalmente lento per scoraggiare attacchi di forza bruta
- Può disabilitare gli utenti dopo alcuni tentativi falliti
- Può utilizzare più di un meccanismo (es: password + one time password)
- Deve essere resistente ad attacchi basati "sul tempo" di risposta

Man-in-the-middle attack:

attacchi basati sull'intercettazione dei dati in rete.

Supponendo che il client X e il server condividano **una chiave segreta K** , ci potrebbe essere sempre un problema **MIM attack**:

se Y ruba il messaggio cifrato trasmesso da X , **può farsi riconoscere dal server come X** -

SOLUZIONE: Messaggi "Nonce"(only once):

Il messaggio viene mandato solo una volta, quindi nel caso in cui un attaccante rubi il messaggio, non potrebbe accedere utilizzando il messaggio.

Reflection attack

Supponiamo che l'autenticazione fra client e server sia bidirezionale, usando lo stesso protocollo:

La **Soluzione** è **eliminare la simmetria aggiungendo il proprio identificativo nei messaggi cifrati.**

Controllo degli accessi agli oggetti protetti

Gli individui autorizzati all'accesso nel sistema **non** hanno diritti omogenei per tutte le risorse.

	File1	File2	File3	Printer	Disk
User1			Read	Write	Read
User2	Read Write	Execute		Write	Read Write

Password di accesso alle risorse

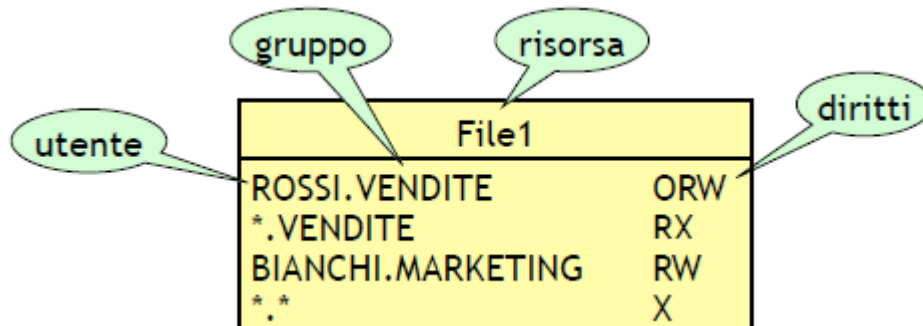
Possibile meccanismo per restringere il diritto di accesso a risorse specifiche, ma:

- poco raffinato (distingue solo fra *possibilità* e *divieto* di accedere);
- poco usabile e flessibile:
 - difficile ricordare una **password diversa per ogni risorsa**;
 - difficile tenere traccia di **chi ha accesso** a risorse specifiche;
 - difficile gestire programmi che accedono a molte risorse;
 - difficile **revocare i diritti**;

Access Control List

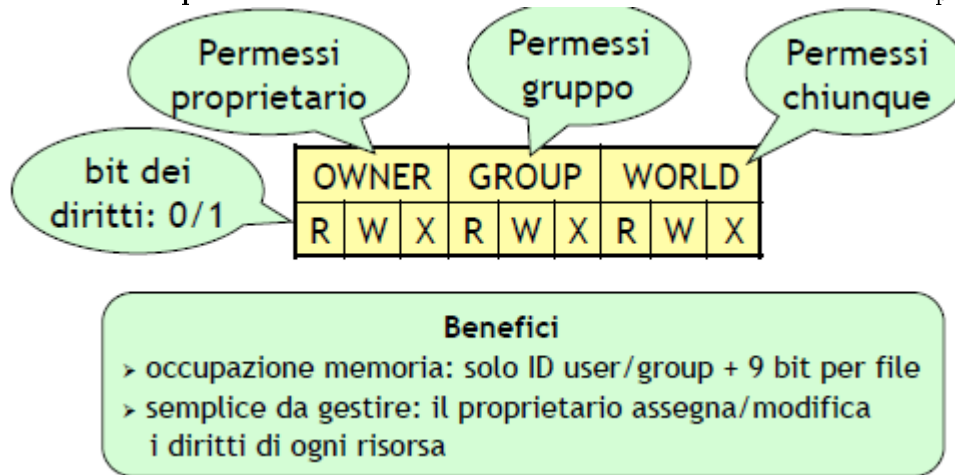
Benefici:

- **Risparmio di spazio** rispetto alla tabella completa;
- Facile **determinare la lista degli utenti** che hanno un diritto specifico su una risorsa condivisa;
- Facile **revocare/modificare l'accesso di un utente** a una risorsa;



Bit di Protezione

Versione semplificata di una Access Control List: usa un numero fisso di bit per ogni file



Covert Channels

Dati sensibili trasmessi su un **canale alternativo**, condiviso (anche legalmente) con un soggetto non autorizzato, in alcuni casi usando segnali non “ovvi”, esempi:

Attaccante: utente interno malevolo o Trojan installato in un programma di servizio;

Meccanismi:

- Scrivendo i dati **su altri file con diversa protezione**;
- Come sopra ma **codificando informazioni nei nomi dei file**;
- **Codifica segreta in file legali**: numero di spazi dopo ‘.’, ultima cifra non significativa di un campo, ... ;
- Sfruttando la presenza/assenza di oggetti in memoria, il blocco/sblocco di un file in un intervallo di tempo concordato, ... ;

Discretionary/Mandatory Access Control

Discretionary Access Control (DAC) (p.e. access control list):

Il proprietario di una risorsa può concedere l'accesso ad altri a sua discrezione;

Mandatory Access Control (MAC):

Il sistema impone un modello che limita e controlla la discrezionalità degli utenti nell'assegnare i diritti di accesso alle risorse;

Modelli di sicurezza MAC

Definiscono in maniera precisa la relazione di accessibilità fra soggetti e oggetti del sistema in base a obiettivi e requisiti di sicurezza specifici, fortemente dipendenti dal dominio applicativo.

Primo esempio: **Sicurezza multi-livello** (nasce in ambienti militari)

Sicurezza multi-livello: confidenzialità:

Classifica i livelli di sicurezza per soggetti e oggetti Accesso consentito solo se:

livello soggetto $\geq$ livello oggetto.

SO trusted: Politiche di controllo degli accessi MAC

Principio:

Se un soggetto rilascia un **oggetto sensibile**, bisogna **gestire opportunamente i casi** in cui la risorsa viene acquisita da un altro soggetto.

Modello di Bell-LaPadula (orientato alla confidenzialità)

Il sistema operativo usa uno schema di controllo accessi di tipo MAC che garantisce **2 proprietà**:

Simple security property (no-read-up):

- un soggetto non legge oggetti di classificazione più alta;

Confinement property (no-write-down):

- un soggetto non scrive oggetti di classificazione più bassa;

Modello BIBA (orientato all'integrità)

Il sistema operativo usa uno schema di controllo accessi di tipo MAC che garantisce **2 proprietà**:

Simple integrity property (no-write-up):

- un soggetto non può modificare oggetti di classificazione più alta;

Integrity Confinement property (no-read-down):

- un soggetto non legge oggetti di classificazione più bassa;

Problemi noti

- Complessità di amministrazione;
- Necessità di implementare applicazioni dedicate;
- Tendenza a forzare "over-classification" delle informazioni;
- Problema del "**blind write-up**" (senza acknowledgement);
- Definire molti livelli di classificazione dei dati può portare a modelli complicati;

Modelli Multi-Lateral

Multi-level security models:

Flussi di informazione controllati in base al livello di criticità dei dati (e degli utenti);

Multi-lateral security models:

Flussi di informazione controllati in base a raggruppamenti semantici dei dati (e degli utenti);

- Compartimenti, Progetti, Gruppi, Ruoli, ...

Modello di Bell-LaPadula esteso con compartimenti

Classifica i livelli di sicurezza per soggetti e oggetti;

Associa le informazioni e i soggetti a "compartimenti";

Accesso consentito solo se:

livello soggetto $\geq$ livello oggetto AND

compartimenti soggetto $\geq$ compartimenti oggetto.

Muraglia Cinese

Obiettivo:

Gestione dei **conflitti di interesse** (COI);

Ovvero: controllare il flusso di dati critici tra aziende concorrenti, limitando l'accesso alle informazioni riservate di una data compagnia da parte di consulenti;

Politica di accesso:

-Organizzazione dei dati a tre livelli:

- Oggetti singoli;
- Dataset aziendali (ogni oggetto in un unico dataset);
- Classi di COI fra dataset (ogni dataset in un'unica classe);

-Regola di accesso per un soggetto S e un oggetto O:

- dataset(O) = dataset(O' che S ha già letto);
- dataset(O) non appartiene a COI(O' che S ha già letto);

Modello di Clark-Wilson(orientato a domini commerciali)

Obiettivo:

Nessuna perdita o corruzione dei dati;

Si basa su 2 principi:

- Principio 1: Well formed transactions

Gli oggetti sensibili sono modificati da un piccolo insieme predefinito di programmi sicuri;

- Principio 2: Separation of duty

Ogni soggetto può eseguire solo parte delle operazioni che partecipano in una transazione;

Auditing e Intrusion Detection

Dati che il S.O può tracciare

- lettura e scrittura di dati: accesso a file, memoria, ecc.
- creazione o distruzione di un oggetto
- accesso degli utenti al sistema: login e logout
- operazioni amministrative: creazioni di utenti, ecc.
- operazioni in modalità privilegiata
- operazioni non usuali
- ...

INTRUSION DETECTION SYSTEMS

Sistemi (automatici e non) che analizzano i dati tracciati (auditing trails) per determinare/prevenire tentativi di compromissione o di uso non autorizzato di un sistema

5 Sicurezza nelle Reti di Computer

Gli **Obiettivi della sicurezza nelle reti** sono gli stessi del caso single host:

- Confidenzialità;
- Integrità;
- Disponibilità;
- Controllo degli accessi;
- Autenticazione/Non ripudio;

Ma, esposizione maggiore ad attacchi maliziosi:

- Un computer in rete è **accessibile da milioni di altri computer** attraverso Internet;
- I **messaggi** scambiati attraverso la rete **attraversano molti nodi intermedi non controllabili**;
- La comunicazione è **gestita da software per cui è più semplice** automatizzare gli attacchi;

Violazioni di sicurezza

- Intercettazione (Confidenzialità);
- Modifica dei dati (Integrità);
- Creazione di traffico (Integrità);
- Denial of service (Disponibilità);

Principali attacchi: Ascolto Intercettazione:

Sniffing:

- Impostando la “Modalità promiscua” si disattiva il filtraggio dei pacchetti a livello della scheda di rete, e si riceve tutto il traffico;
- I pacchetti intercettati contengono le intestazioni dei protocolli (es: indirizzi IP, porte TCP) e i dati applicativi (es: dati utente);

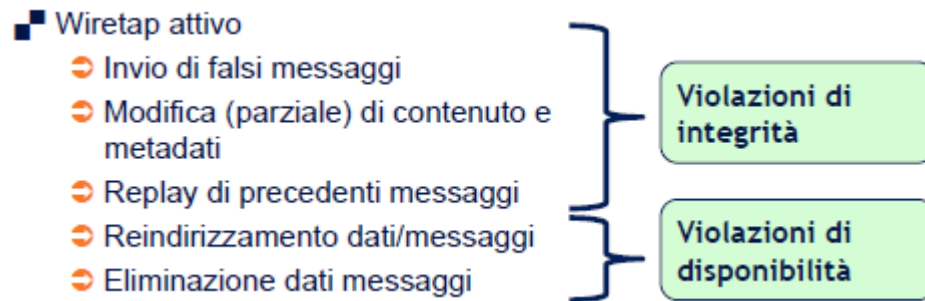
Wiretap passivo: Analisi del traffico di rete

- Tentativo di inferire intelligenza dai messaggi che fluiscono sulla rete indipendentemente dal loro contenuto;
- L'attaccante studia i metadati (es: indirizzi sorgente e destinazione) piuttosto che al contenuto specifico dei messaggi;

■ Wiretap passivo

- Intercettazione attraverso sniffing dei pacchetti che transitano in rete (es. su LAN non-switched, impostando scheda di rete in modalità promiscua)
- Rough wireless access point
- Mezzi fisici: “lettura” per induzione elettromagnetica, alterazione del cavo, intercettazione di comunicazione wireless/satellite mediante antenne
- Analisi del traffico di rete: l'obiettivo non sono i dati, ma i metadati di una comunicazione

Violazioni di
confidenzialità



Principali attacchi: Violazioni Autenticazione

- Attacco ovvio se l'autenticazione non è richiesta
- Indovinando la password (Password facili, Attacchi di forza bruta);
- Intercettazione della password trasmessa in chiaro, o intercettazione password cifrata ma riusabile;
- Password che fanno fallire il meccanismo di autenticazione (sfruttando debolezze dell'implementazione);

Principali attacchi: Falsificazione di identità

Spoofing: Falsificazione di identità (per invio/furto di informazioni):

- Pharming: nodo della rete che impersona un altro nodo;
- Phishing: spesso passo preliminare di un attacco pharming;
- IP Spoofing;
- Fake public key identity: per realizzare wiretap attivo/passivo in protocolli che fanno uso di crittografia;
- Hijacking di sessione;

Queste sopra sono Tecniche utili in attacchi per realizzare violazioni di sicurezza.

Principali attacchi: Violazioni Autenticazione

Host che si finge un altro host (in cui gli utenti hanno fiducia).

Di solito l'attaccante cerca di **replicare il medesimo scenario applicativo dell'host vero** (esempio: riproducendo la stessa grafica di un sito web) in modo da non far notare il "**camuffamento**";

Modalità:

- Sfruttando confusione fra URL con nomi simili;
- Alterando le tabelle di un DNS (server dei nomi di dominio);
- Sfruttando **phishing**;

IP Spoofing:

- Creazione di traffico IP con un indirizzo di sorgente falso, (es. un indirizzo che non ha bisogno di autenticazione);

Spoofing: Fake public key identity

Principali attacchi: Denial of Service (DoS)

Attacchi fisici a computer e apparati di rete (solitamente prevedibili o difficili da attuare):

- **Flooding:**
 - Syn flooding su TCP/UDP;
 - Flooding attraverso protocolli ICMP come "ping";
 - **Smurf:** Ping broadcast + IP spoofing;
- Attacchi **DNS:** Reindirizzamento del traffico attraverso modifiche su server dei nomi;
- Attacchi **DDoS** (Distributed DoS): cavalli di troia (zombie) + Flooding;

Social Engineering

Furba e intelligente manipolazione della naturale tendenza umana alla fiducia (molti attacchi avvengono per telefono).

Esempi:

- Ottenere accesso non autorizzato ad un edificio sostenendo di aver perso il proprio badge;
- Ottenere informazioni riservate per telefono fingendosi un'altra persona;

Difese contro attacchi dalla rete

Firewall

HW/SW per **controllare il traffico in ingresso/uscita di una rete fidata** rispetto a reti non sicure;

Tipi di Firewall

Packet Filter:

- Livello di Azione: protocollo di rete;
- Monitoraggio su: header IP dei pacchetti;

Application Gateway:

- Livello di Azione: protocolli applicativi;
- Monitoraggio su: dati applicativi;

Packet Filtering

Router che esamina i pacchetti in transito ed esegue una selezione in base allo header IP dei pacchetti;

Esempi:

- Bloccare il traffico verso porte (servizi) o proveniente da host pericolosi o non-proveniente da host fidati;
- Bloccare pacchetti malformati (difesa da attacchi DoS);
- Non ammettere pacchetti esterni con un indirizzo sorgente interno alla rete (difesa contro IP spoofing);

Application Gateway (Proxy)

Agisce a livello applicativo per **monitorare il traffico** fra applicazioni.

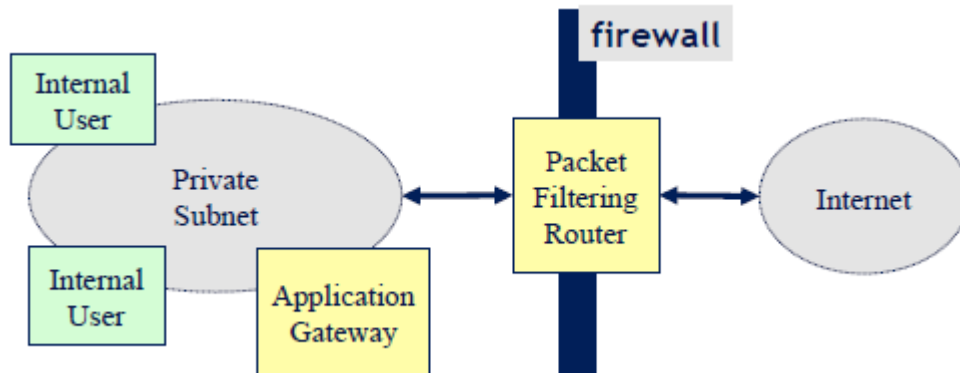
Per comunicare con un'applicazione interna alla rete protetta bisogna **connettersi al firewall** (che fa da proxy per l'applicazione).

A sua volta il firewall si **connette all'applicazione interna** e replica (**relay**) i pacchetti fra le due connessioni.

Si possono configurare firewall (proxy) diversi per diversi protocolli applicativi;

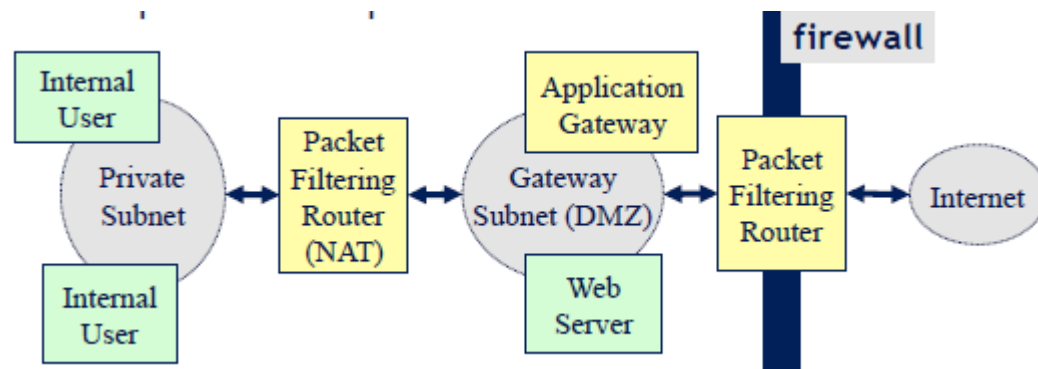
Soluzione Mista 1: Screened Subnet

Per certi servizi, il router accetta solo pacchetti **da/e/per** il gateway sulle porte ammesse. Il gateway controlla il traffico applicativo.



Soluzione Mista 2: DMZ

- DMZ = **zona demilitarizzata**;
- Utenti e gateway su reti fisicamente separate;
- Due livelli di filtraggio;
- Servizi pubblicamente accessibili nella DMZ completamente separati dalla rete interna;



Protocolli di rete con crittografia

SSL: Secure Socket Layer

Aggiunge **privacy e integrità** ai pacchetti in transito su una connessione **TCP-IP**.

Evoluzione verso **TSL** (Transport Layer Security) con la standardizzazione **IETF**.

SSL (TLS) modifica il pacchetto applicativo e aggiunge informazioni per **permettere alla macchina di destinazione di riconoscere l'informazione**.

Architettura SSL

SSL Handshake permette di **stabilire** la connessione e **negoziare** il livello di sicurezza (es. algoritmi crittografici da usare);

SSL Record Protocol implementa lo **scambio sicuro dei pacchetti** secondo i parametri crittografici stabiliti.

SSL Handshake

Usato per stabilire la **connessione sicura** fra un **SSL client** e un **SSL server**.

Precede lo scambio di dati relativo alla sessione applicativa (es. HTTP).

Funzioni di SSL Handshake:

- Negoziazione della **versione del protocollo**;
- Negoziazione degli **algoritmi crittografici** per **privacy** (es. DES) e **integrità** (es. MD5);
- Autenticazione del client (opzionale);
- Autenticazione del server (opzionale);
- Condivisione di una chiave comune (scambiata attraverso crittografia a chiave pubblica);
- **Verifica d'integrità** dei parametri concordati;

Fasi Handshake

- **Hello Phase**, dati scambiati con i messaggi di **hello**;
- **Server Authentication**:
 - certificate**: Invio del certificato del server (es. X.509);
 - server_public_key**: Se non si è mandato il certificato o se la chiave nel certificato vale solo per le firme;
 - certificate_request**: Il server richiede l'autenticazione del client;
- **Client Authentication**:
 - certificate**: Invio del certificato del client (es. X.509);
 - session_key**: Chiave segreta, cifrata con la chiave pubblica del server;
 - certificate_verification**: Conferma che il certificato fornito dal server risulta verificato;
- **Finish**:
 - change_cypher_spec**: Conferma che le impostazioni di sicurezza sono state settate secondo la negoziazione effettuata;
 - finished**: Terminazione handshake, pronto a trasmettere;

Sicurezza nelle Applicazioni Web

HTTPS = HTTP su SSL/TSL:

HTTPS agisce sulla **porta TCP 443** (HTTP porta 80);

I componenti coinvolti devono implementare SSL;

- Browser;
- Server Web;
- Eventuali Proxy Server;

Limiti di SSL

SSL (TLS) protegge la comunicazione fra applicazioni, **aggiungendo sicurezza** sopra lo strato di trasporto. Le informazioni di rete sono in chiaro.

Vulnerabilità:

- Spoofing degli indirizzi IP;
- Analisi del traffico di rete;

IPsec

Sicurezza a livello di rete

Implementazione di privacy e integrità a livello IP:

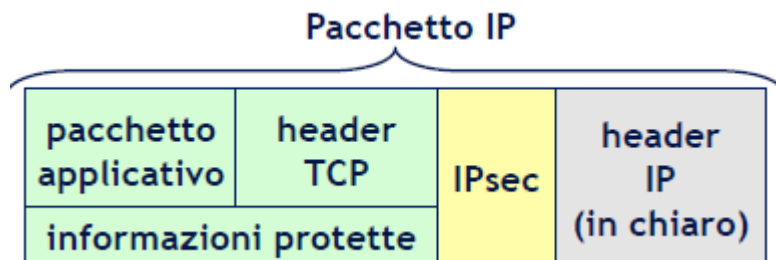
- trasparente alle applicazioni e agli utenti;
- host-to-host;
- anche con protocolli di trasporto connection-less;

Modalità di Funzionamento

+Modalità **trasporto**;

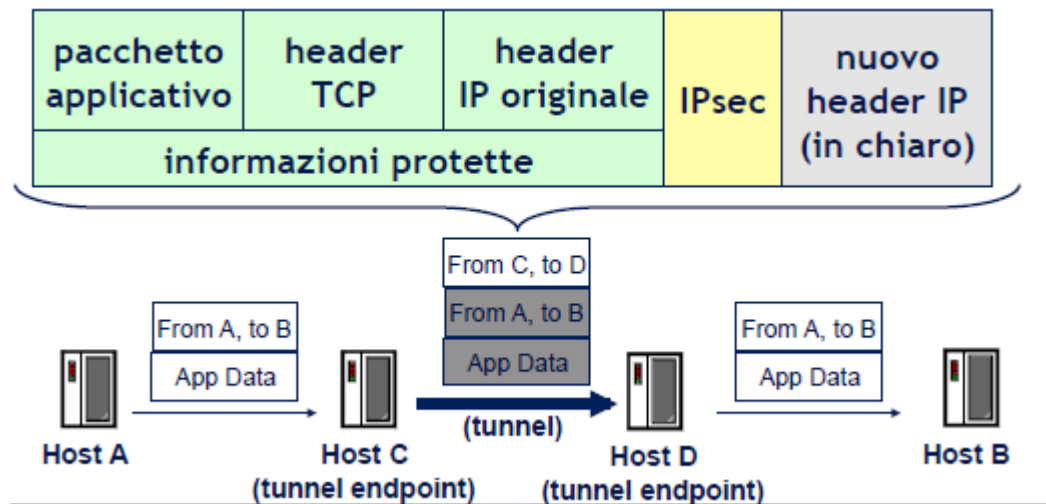
Protezione simile a SSL, ma:

- host-to-host
- trasparente alle applicazioni;



+Modalità **Tunneling**;

Un ulteriore header **incapsula i pacchetti per l'instradamento** fra i due host (tunnel end-points), proteggendo le informazioni di rete originali;



Virtual Private Networks (VPN)

Reti private attraverso connessioni **insicure**:

- Tutto il traffico sulle connessioni insicure è cifrato;
- Es. lavoro remoto, collegamento sicuro alla rete aziendale;
- Es. dipartimenti remoti connessi attraverso reti pubbliche;

Uso di IPsec

Protezione del traffico fra **due host**;

Protezione del traffico fra **due sottoreti**; (protezione del traffico fra i **gateway delle sottoreti**);

- Protezione del traffico fra **un host e una sottorete**; (protezione del traffico fra **l'host e il gateway della sottorete**);

Difese contro attacchi di rete

-Maggiore resistenza a **sniffing e modifica dei dati**:

- Usare IPsec anche con altri protocolli di crittografia;

-Maggiore protezione **contro traffic analysis**:

- Uso della crittografia a livello di rete per tutti i messaggi;
- Costruire pochi tunnel fra sottoreti invece che molti tunnel per comunicazioni fra macchine specifiche;

-Maggiore protezione **contro IP spoofing**:

- anche l'indirizzo IP d'origine può essere autenticato;